Exploratory Data Analysis (EDA)

Detailed Answers to 16-Mark Questions

Question 1 (16 Marks)

Dataset: *Customer purchase data (Customer_ID, City, Category, Amount). Tasks: Group by City and total purchase; Aggregate amount category-wise; Create pivot table (City vs Category); Explain how transformed data supports EDA.*

1. Group by City – Total Purchase Amount

Using `groupby('City')['Amount'].sum()` we obtain the total purchase value for each city:

City	Total Amount (Rs.)
Chennai	97,000
Coimbatore	30,000
Madurai	25,000

Chennai contributes the highest total purchase amount (Rs.97,000), followed by Coimbatore and Madurai. This immediately highlights the dominant market.

2. Aggregate Amount Category-wise

Category	Total Amount (Rs.)
Electronics	75,000
Furniture	47,000
Clothing	30,000

Electronics is the top revenue category (Rs.75,000). Aggregation reduces six individual transactions into three clear category totals, making ranking and comparison trivial.

3. Pivot Table – City vs Category Purchase Amount

City	Clothing	Electronics	Furniture	Total
Chennai	–	75,000	22,000	97,000
Coimbatore	30,000	–	–	30,000
Madurai	–	–	25,000	25,000

The pivot table (created with `pd.pivot_table(..., index='City', columns='Category', values='Amount', aggfunc='sum')`) cross-tabulates the two dimensions and reveals that Electronics sales are concentrated solely in Chennai, while Clothing is exclusive to Coimbatore and Furniture appears in both Chennai and Madurai.

4. How the Transformed Dataset Supports Exploratory Analysis

• **Dimensionality reduction:** Six rows become compact summaries, exposing city- and category-level patterns without scanning every record.
• **Pattern detection:** Dominance of Chennai and Electronics becomes obvious; missing combinations (e.g., Clothing in Chennai) are visible as blanks.
• **Further analysis ready:** The aggregated and pivoted data can feed bar charts, heatmaps, or ranking analyses and support business questions such as “Which city–category pair should receive more inventory?”.
• **Scalability:** The same grouping/pivoting logic works unchanged on thousands of transactions, making EDA practical for large datasets.

Question 2 (16 Marks)

Datasets: *Product (Product_ID, Product, Category) and Sales (Product_ID, Region, Sales). Tasks: Merge on Product_ID; Group by Category & Region; Calculate total sales; Construct pivot table.*

1. Merge the Datasets using Product_ID

An inner merge on `Product_ID` produces a unified table containing product attributes together with regional sales figures:

Product_ID	Product	Category	Region	Sales
P101	Laptop	Electronics	North	1,50,000
P101	Laptop	Electronics	South	1,20,000
P102	Chair	Furniture	South	45,000
P103	Shirt	Clothing	North	25,000

```
Code: pd.merge(product_df, sales_df, on='Product_ID', how='inner')
```

2. Group by Category and Region & 3. Calculate Total Sales

Category	Region	Total Sales
Electronics	North	1,50,000
Electronics	South	1,20,000
Furniture	South	45,000
Clothing	North	25,000

Grouping with `groupby(['Category','Region'])['Sales'].sum()` yields the above totals. Electronics dominates in both regions; Clothing and Furniture have lower, region-specific sales.

4. Pivot Table Summarizing Results

Category	North	South	Grand Total
Electronics	1,50,000	1,20,000	2,70,000
Furniture	–	45,000	45,000
Clothing	25,000	–	25,000
Total	1,75,000	1,65,000	3,40,000

The pivot table provides a compact cross-tabulation that supports rapid comparison of category performance across regions and clearly shows that Electronics accounts for the majority of revenue (Rs.2,70,000 out of Rs.3,40,000).

Question 3 (16 Marks)

Dataset: *Monthly temperature (°C) from Jan to Aug. Tasks: Select visualizations; Analyse seasonal patterns; Role of labels/legends/annotations; Evaluate histograms.*

1. Appropriate Visualizations

- **Line plot** – best primary choice: clearly shows the continuous trend of temperature across months.
- **Bar chart** – useful alternative for comparing discrete monthly values side-by-side.
- **Area plot** – can emphasize the magnitude of temperature change over the season.

2. Seasonal Pattern Analysis

Temperatures rise steadily from January (28 °C) to a peak in May (38 °C), then decline through June–August (35 → 31 °C). This reflects a typical pre-monsoon heating followed by the onset of cooler conditions. A line plot with markers at each month makes the rise-and-fall pattern immediately visible and quantifiable (peak-to-trough drop of 7 °C from May to August).

3. Role of Labels, Legends and Annotations

• **Labels** (title, x-axis “Month”, y-axis “Temperature (°C)”) give the plot context and units so the reader does not have to guess meaning.
• **Legend** is useful if multiple series (e.g., actual vs. long-term average) are shown on the same axes.
• **Annotations** (text arrows or call-outs) can highlight the May peak (“Hottest month: 38 °C”) or the August decline, directing attention to the most important features and improving interpretability for non-technical audiences.

4. Are Histograms Suitable?

Histograms are **not the most suitable** visualization for this dataset. A histogram shows the frequency distribution of a continuous variable and discards the temporal ordering of the months. Because the primary analytical goal is to understand *how temperature changes over time*, a line or bar chart that preserves the month sequence is far more informative. A histogram could be used only as a secondary tool to examine the overall shape of the temperature values (e.g., whether they are roughly symmetric), but it would not reveal seasonal patterns.

Question 4 (16 Marks)

Dataset: *Customer ages (C1–C6: 22, 26, 31, 35, 40, 45). Tasks: Analyse age distribution; Numerical summaries; Usefulness of Seaborn; Justify visualization choice.*

1. Age Distribution Analysis

The six ages range from 22 to 45 and increase roughly linearly. Suitable visualizations:
• **Histogram / KDE plot** – shows the shape of the distribution (slightly right-skewed or nearly uniform given the small sample).
• **Box plot** – displays median, quartiles and any potential outliers.
• **Strip / swarm plot** – shows every individual observation without binning, ideal for n = 6.

2. Numerical Summaries

Statistic	Value
Count	6
Mean	33.17
Median	33
Std. Deviation	≈ 8.6
Minimum	22
Maximum	45
Range	23

Mean ≈ median suggests a roughly symmetric distribution. The range of 23 years indicates moderate spread; no extreme outliers are present.

3. Usefulness of Seaborn Plots

Seaborn provides high-level functions that automatically compute and display statistical summaries:
• `sns.histplot(..., kde=True)` overlays a kernel-density estimate on the histogram.
• `sns.boxplot()` and `sns.violinplot()` give compact five-number summaries with attractive defaults.
• `sns.swarmplot()` shows every point without overlap.
These plots require far less code than pure Matplotlib while producing publication-quality aesthetics, making Seaborn especially valuable for rapid EDA.

4. Justification of Visualization Method

For a small numeric sample the combination of a **box plot + swarm plot** (or a histogram with KDE) is optimal: the box plot communicates central tendency and spread in a single glance, while the swarm plot retains every individual age, avoiding information loss caused by binning. This dual view satisfies both summary and detail needs of exploratory analysis.

Question 5 (16 Marks)

Dataset: *Student marks in Python, Statistics, EDA (6 students). Tasks: Best visualization for Python distribution; Compare three subjects; Role of legends/colors/labels/annotations; Seaborn vs Matplotlib choice.*

1. Most Suitable Visualization for Python Marks Distribution

A **histogram** (or histogram with KDE) is the most appropriate choice for examining the distribution of Python marks. It shows the frequency of marks across score ranges, reveals shape (skewness, modality) and highlights possible clusters or gaps. With only six observations a strip/swarm plot can also be useful as a complementary view that retains every individual score.

2. Comparing Performance of the Three Subjects

Effective comparative plots include:
• **Grouped bar chart** – side-by-side bars for each student across the three subjects.
• **Box plots** (one per subject) – compare medians, IQRs and outliers across subjects.
• **Radar / parallel-coordinates plot** – show multivariate profile of each student.
• **Heatmap** of the marks matrix – quick visual ranking of high/low scores.
From the data, student S4 scores highest overall; S3 is the weakest. Statistics marks tend to be slightly lower than Python and EDA for most students.

3. Improving Interpretation with Legends, Colors, Labels & Annotations

• **Legends** distinguish the three subjects when multiple series share the same axes.
• **Colors** (distinct, color-blind-friendly palette) make each subject instantly recognizable and can encode performance level (e.g., green = high, red = low).
• **Labels** (axis titles with units, informative plot title, student IDs on the x-axis) supply necessary context.
• **Annotations** can mark the highest/lowest scorer or a class-average line, drawing attention to key insights without forcing the reader to calculate them.

4. Seaborn vs Matplotlib – Justification

For this analysis **Seaborn is more appropriate** as the primary tool. It offers ready-made statistical plots (boxplot, violinplot, histplot with KDE, heatmap) with sensible aesthetic defaults and less boilerplate code. Matplotlib remains useful for fine-grained customization or when a highly tailored annotation layout is required, but the exploratory goals of distribution comparison and multi-subject ranking are achieved more efficiently with Seaborn.

Question 6 (16 Marks)

Dataset: *Retail sales (Product, Category, Region, Quantity, Sales). Tasks: Group by Category (total Sales & Quantity); Pivot Category-wise Sales by Region; Identify highest-selling category; Explain support for business decisions.*

1. Group by Category – Total Sales and Quantity

Category	Total Quantity	Total Sales (Rs.)
Electronics	15	4,30,000
Furniture	12	1,40,000
Clothing	22	66,000

Electronics generates by far the highest revenue despite a moderate quantity; Clothing has the highest quantity but lowest revenue.

2. Pivot Table – Category-wise Sales for each Region

Category	North	South	Total
Electronics	2,50,000	1,80,000	4,30,000
Furniture	80,000	60,000	1,40,000
Clothing	36,000	30,000	66,000

3. Highest-Selling Category (Aggregation)

Applying `groupby('Category')['Sales'].sum().idxmax()` (or sorting the aggregated Series) identifies **Electronics** as the highest-selling category with Rs.4,30,000 total sales.

4. Supporting Business Decision-Making

The transformed data enables concrete decisions:
• **Inventory & procurement:** Prioritize Electronics stock, especially in the North region.
• **Marketing spend:** Allocate larger budgets to Electronics campaigns; investigate why Clothing volume is high but revenue low (possible price or margin issues).
• **Regional strategy:** North consistently outperforms South across categories—consider expanding North operations or diagnosing South under-performance.
• **Product-mix optimization:** Furniture and Clothing contribute less; management may decide to promote higher-margin items or phase out low contributors.
Aggregated and pivoted views turn raw transaction rows into actionable KPIs that non-technical stakeholders can immediately understand.

Question 7 (16 Marks)

Dataset: *Product, Region, Quantity, Sales (Laptop/Mobile/Tablet in North/South). Tasks: Group by Region (total Sales); Group by Product (Quantity & Sales); Pivot Product-wise Sales by Region; Explain use for further EDA.*

1. Group by Region – Total Sales

Region	Total Sales (Rs.)
North	4,78,000
South	3,90,000

North leads with Rs.4,78,000 versus South's Rs.3,90,000.

2. Group by Product – Total Quantity and Sales

Product	Total Quantity	Total Sales (Rs.)
Laptop	8	4,00,000
Mobile	16	2,88,000
Tablet	12	1,80,000

Laptop is the revenue leader; Mobile has the highest unit volume.

3. Pivot Table – Product-wise Sales for each Region

Product	North	South	Total
Laptop	2,50,000	1,50,000	4,00,000
Mobile	1,08,000	1,80,000	2,88,000
Tablet	1,20,000	60,000	1,80,000

4. Use of Aggregated Results for Further Exploratory Analysis

• Rank products and regions to focus subsequent deep-dives (e.g., why Mobile sells better in South).
• Compute derived metrics such as average selling price (Sales/Quantity) per product–region cell.
• Feed the pivot table into heatmaps or clustered bar charts for visual pattern detection.
• Identify candidates for correlation or regression analysis (e.g., relationship between Quantity and Sales).
• Serve as a clean input for forecasting models or what-if scenario planning.
Thus the aggregated views act as a bridge between raw data and more advanced statistical or machine-learning exploration.

Question 8 (16 Marks)

Datasets: *Student (Student_ID, Name, Department) and Marks (Student_ID, Python, Statistics). Tasks: Merge on Student_ID; Reshape merged data; Average marks by department; Cross-tabulation of Department vs performance category.*

1. Merge on Student_ID

Student_ID	Name	Department	Python	Statistics
101	Arun	AI&DS;	85	78
102	Banu	CSE	72	88
103	Charan	AI&DS;	91	84
104	Divya	CSE	68	75

```
Code: pd.merge(student_df, marks_df, on='Student_ID')
```

2. Reshape the Merged Dataset

Melting the wide marks columns into long format produces a structure suitable for many Seaborn plots and group-by operations:

```
melted = merged.melt(id_vars=['Student_ID','Name','Department'],
value_vars=['Python','Statistics'], var_name='Subject', value_name='Marks')
```

Resulting columns: Student_ID, Name, Department, Subject, Marks – one row per student–subject combination.

3. Average Python and Statistics Marks by Department

Department	Avg Python	Avg Statistics
AI&DS;	88.0	81.0
CSE	70.0	81.5

AI&DS; students outperform CSE students in Python; Statistics averages are nearly identical.

4. Cross-Tabulation – Department vs Performance Category

First create a performance category (e.g., High ≥ 80, Medium 65–79, Low < 65) on average or on a chosen subject, then:

```
pd.crosstab(merged['Department'], merged['Performance'])
```

Example result (using overall average mark):

Department	High	Medium	Low
AI&DS;	2	0	0
CSE	1	1	0

The cross-tabulation quickly shows that both AI&DS; students fall in the High category while CSE is mixed, supporting department-level performance comparisons.

Question 9 & 10 (16 Marks) – Identical Questions

Dataset: *Student marks in Python, Statistics, EDA (8 students A–H). Tasks: Best visualization for Python distribution; Use of subplots to compare subjects; Role of legends/colors/titles/annotations; Usefulness of a 3-D plot.*

1. Most Appropriate Visualization for Python Marks Distribution

A **histogram with an overlaid kernel-density estimate (KDE)** is the most appropriate choice. It reveals the shape of the distribution (approximate normality, possible mild left-skew), the location of the centre, and the spread. With eight observations a complementary **box plot** or **swarm plot** can also be shown to retain individual values. Justification: histograms/KDEs are the standard EDA tool for univariate continuous distributions and directly answer “How are Python marks spread?”.

2. Using Subplots to Compare the Three Subjects

Subplots arranged in a 1×3 or 2×2 grid allow simultaneous visual comparison:
• Three side-by-side histograms/KDEs (one per subject) share a common x-axis scale so differences in centre and spread are obvious.

• Three box plots in a single figure highlight median differences and outlier presence.
• A single figure containing three violin plots or a grouped bar chart of means also works.
Matplotlib's `plt.subplots()` or Seaborn's `FacetGrid` / `sns.boxplot(..., col=...)` make such layouts straightforward. Side-by-side placement eliminates the need to hold multiple separate plots in memory and supports direct visual ranking of subjects.

3. Improving Interpretation with Legends, Colors, Titles and Annotations

• **Titles** (overall figure title + subplot titles) orient the reader immediately.
• **Colors** differentiate subjects or encode score magnitude (sequential colormap).
• **Legends** identify series when multiple distributions share axes.
• **Annotations** (mean/median lines, text labels for highest/lowest scorers, percentage of students above a threshold) surface key statistics that would otherwise require mental calculation, thereby accelerating insight generation.

4. Usefulness of a Three-Dimensional Plot

A 3-D scatter plot with axes Python–Statistics–EDA can in principle show the joint relationship among the three marks. However, for only eight points the added visual complexity rarely justifies the benefit: depth perception is limited on a 2-D screen, occlusion occurs, and interactive rotation is needed for full understanding. More effective alternatives are a **pair-plot** (all pairwise 2-D scatter plots) or a **parallel-coordinates plot**. Consequently a static 3-D plot is of limited practical usefulness for this small multivariate dataset; pairwise 2-D views remain preferable.

Question 11 (16 Marks)

Scenario: *E-commerce transaction data (customer ID, product, category, price, quantity, purchase date). Tasks: Group/aggregate for category-wise sales; Suggest pivot-table structure; Suitable Matplotlib plots for transaction-value distribution; How Seaborn visualizes relationships.*

1. Grouping and Aggregating for Category-wise Sales

First compute a line-total column if needed: `df['Revenue'] = df['price'] * df['quantity']`. Then aggregate:

```
category_sales = df.groupby('category').agg(Total_Revenue=('Revenue','sum'),
Total_Quantity=('quantity','sum'), Avg_Price=('price','mean'),
Transaction_Count=('customer ID','count'))
```

This yields, for every product category, total revenue, units sold, average unit price and number of transactions—core KPIs for category performance analysis.

2. Appropriate Pivot-Table Structure

A useful multi-dimensional summary is:

```
pd.pivot_table(df, index='category', columns=pd.Grouper(key='purchase date',
freq='M'), values='Revenue', aggfunc='sum', fill_value=0)
```

Rows = categories, columns = months (or weeks), cells = total revenue. Alternative structures: Category × Customer-segment, or Category × Price-band. Such pivots expose seasonality, category growth trajectories and cross-category comparisons in a single table.

3. Suitable Matplotlib Plots for Transaction-Value Distribution

• **Histogram** (`plt.hist`) of the Revenue (or price) column – primary tool for shape, skewness and outliers.
• **Box plot** – five-number summary and outlier detection, especially useful when stratified by category.
• **Empirical cumulative distribution (ECDF)** – shows the proportion of transactions below any given value.
• Log-scaled axes are often helpful because e-commerce revenue distributions are typically right-skewed.

4. Using Seaborn to Visualize Relationships among Variables

• `sns.scatterplot(x='price', y='quantity', hue='category', size='Revenue')` – price–quantity relationship coloured by category.
• `sns.boxplot(x='category', y='Revenue')` or `sns.violinplot` – distribution of revenue per category.
• `sns.heatmap` of a correlation matrix (price, quantity, revenue, day-of-week, etc.).
• `sns.pairplot` or `sns.jointplot` for pairwise numeric relationships.

• `sns.lineplot` with date on the x-axis to reveal temporal purchasing patterns.
Seaborn's statistical estimation (regression lines, confidence intervals, KDE) and automatic multi-plot faceting accelerate the discovery of associations that pure Matplotlib would require more code to surface.

Question 12 (16 Marks) – Geographical / City Administration Dataset

Dataset: *Latitude, longitude, population and pollution levels for different locations. Objective: explore and communicate geographical patterns. Tasks: Prepare data for geo-visualization; Appropriate map visualization; Use of colors & annotations for pollution; Value of combining geographic views with other EDA plots.*

1. Preparing Data for Geographical Visualization

• Ensure latitude and longitude are numeric and within valid ranges (–90…90, –180…180).
• Handle missing coordinates (drop or impute) and remove obvious outliers.
• Optionally project coordinates into a local CRS if distance calculations are needed.
• Create derived columns such as pollution category (Low/Medium/High) by binning the continuous pollution variable.
• Normalize or log-transform population if marker sizes will represent population, to avoid extreme size differences.
• Merge any external boundary files (city wards, administrative polygons) if choropleth maps are planned.

2. Appropriate Visualization Approach for Locations on a Map

• **Scatter map** (points plotted at lat/lon) using libraries such as Matplotlib's Basemap/Cartopy, Folium, or Plotly – simplest and most flexible for point data.
• **Bubble map** – marker size encodes population, colour encodes pollution level.
• **Choropleth map** – if polygon boundaries are available, colour each administrative unit by average pollution or total population.
• Interactive maps (Folium/Leaflet or Plotly) allow zooming and hover tooltips, which are especially valuable for dense urban datasets.

3. Using Colors and Annotations to Communicate Pollution Levels

• Apply a sequential or diverging colormap (e.g., green → yellow → red) so that higher pollution is instantly associated with "warmer" colours.
• Provide a clear colour bar / legend with numeric breakpoints.
• Annotate critical locations (highest pollution, densest population, monitoring stations) with text labels or call-out arrows.
• Optionally overlay contour lines or a semi-transparent heatmap layer to show pollution gradients between discrete measurement points.
These visual encodings turn a raw table of numbers into an immediately interpretable spatial story for decision makers.

4. Usefulness of Combining Geographic Visualization with Other EDA Plots

Geographic maps answer "Where?"; classical EDA plots answer "How much?", "How distributed?" and "How related?". Combining both yields richer insight:
• A map showing pollution hotspots can be paired with a histogram of pollution values and a scatter plot of pollution versus population to test whether denser areas are systematically more polluted.
• Box plots of pollution by administrative zone complement a choropleth of the same zones.
• Time-series line plots of pollution at selected map locations reveal temporal dynamics that a static map cannot show.
• Correlation heatmaps among pollution, population density, traffic volume, etc., gain spatial context when the same variables are displayed on a map.
Thus the geographic layer and the statistical layer reinforce each other, producing a more complete exploratory narrative and supporting evidence-based urban-policy decisions.